

AI Utility Report

Part 6, Task B — Test-Run Triage Assistant

Author: Uma Uka | Date: 31 August 2026

1. Use Case Chosen

The brief offered three example use cases. I built “**summarize a failed test run's logs into a triage note**” — chosen over the other two specifically because it let me use real data already in this repo (Part 2's own Playwright JSON reports, including the actual `test.fail()`/BUG-n annotations) rather than a fabricated bug report or invented wallet-transfer edge cases.

The script, `ai-utility/triage-report.mjs`, reads a Playwright JSON report and produces a Markdown triage note that separates: genuinely new/unexpected failures (need a human now), a tracked bug that unexpectedly *passed* (may be fixed — verify and promote out of `test.fail()`), known tracked bugs still behaving as documented (no action needed), and flaky tests.

2. Prompt Design

Key design choice: categorization is done **deterministically in code** from the report's own status/annotation fields — the LLM never counts or classifies. Its only job is to write a short human-readable summary and suggest a likely root-cause area and priority for each already-identified genuine failure. This directly follows what my own Part 6 Task A critique found: LLMs are strong at qualitative judgment and weak at exact arithmetic/bookkeeping — so the design keeps the model out of the bookkeeping entirely.

Output is forced into a fixed schema using **strict tool use** (`strict: true` on the tool definition, `tool_choice` forcing that exact tool) rather than asking for free-text JSON — this guarantees schema-valid output directly from the API, with no markdown-fence or parse-error handling needed.

System prompt (verbatim):

```
You are a QA triage assistant. You will receive a JSON summary of one Playwright test run.
Known, already-tracked bugs and possibly-fixed bugs are provided for context only -- they are
already handled by the caller and need no analysis from you. Your only job is: for each entry
in `unexpected_failures`, suggest a short likely_area, a one-sentence likely_cause hypothesis
based on its error_message, and a suggested_priority. Also write a 2-4 sentence executive_summary
of overall run health. You MUST return exactly one entry in unexpected_failures for every entry
given in the input, matching test_title and file exactly -- never invent, merge, or omit an entry.
```

Tool schema (abridged — full version in the script):

```
{
  "name": "submit_triage_analysis",
  "strict": true,
  "input_schema": {
    "type": "object",
    "properties": {
      "executive_summary": { "type": "string" },
      "unexpected_failures": { "type": "array", "items": {
        "properties": {
          "test_title": {"type": "string"}, "file": {"type": "string"},
          "likely_area": {"type": "string"}, "likely_cause": {"type": "string"},
          "suggested_priority": {"enum": ["P1", "P2", "P3"]}
        },
        "required": [...], "additionalProperties": false
      }},
      "flaky_tests_note": { "type": "string" }
    },
    "required": ["executive_summary", "unexpected_failures", "flaky_tests_note"],
    "additionalProperties": false
  }
}
```

3. Example Input and Output

No live API call was made for this example — no Anthropic API key was available in this environment. The script has an explicit, clearly-banner-labeled `--mock` mode that runs the entire real pipeline (parse → categorize → validate → render) end to end with a deterministic stand-in for the API call, so every step below except the actual model call is genuine, executed output — not a hand-written example. The exact request shown in Section 2 is what would be sent to `claude-opus-5` on a real run.

Input: a hand-built sample report (`ai-utility/sample-reports/mixed-run.json`) covering all four categories in one run, since the real suite currently has zero genuine unexpected failures. This is the condensed JSON the script actually builds and would send as the prompt:

```
{
  "stats": {"expected": 2, "skipped": 0, "unexpected": 2, "flaky": 1, "duration": 842.5},
  "unexpected_failures": [
    {"test_title": "creates a token with valid credentials", "file": "auth.spec.js",
     "error_message": "Error: connect ECONNREFUSED 127.0.0.1:3001"}
  ],
  "possible_fixed_bugs": [
    {"test_title": "deleting an already-deleted booking returns 404", "file": "crud.spec.js",
     "bug": "BUG-2: DELETE on a missing booking returns 405 instead of 404"}
  ],
  "flaky": [
    {"test_title": "GET /booking filtered by checkin/checkout date range ...",
     "file": "filtering.spec.js"}
  ]
}
```

Output: the actual Markdown produced by `node ai-utility/triage-report.mjs ai-utility/sample-reports/mixed-run.json --mock`:

```
# Test Run Triage Note

**Run summary:** 2 expected, 2 unexpected, 1 flaky, 0 skipped (0.8s)

## Summary
[MOCK] 1 unexpected failure(s) detected alongside 1 known tracked issue(s)
behaving as documented.

## Needs Human Attention Now
### creates a token with valid credentials
- File: auth.spec.js
- Error: Error: connect ECONNREFUSED 127.0.0.1:3001
- Likely area: network/connectivity
- Likely cause: [MOCK] Error message suggests the service under test was unreachable.
- Suggested priority: P1

## Possibly Fixed -- Verify & Promote Out of test.fail()
- deleting an already-deleted booking returns 404 (crud.spec.js) now passes despite
  being tracked as: BUG-2: DELETE on a missing booking returns 405 instead of 404

## Known, Tracked -- No Action Needed
- BUG-1: DELETE /booking/{id} returns 201 Created instead of 200/204

## Flaky -- Needs Investigation
- GET /booking filtered by checkin/checkout date range ... (filtering.spec.js)
[MOCK] Flaky tests present; investigate separately.
```

4. Validation Approach

Three layers, applied in order, before any output is trusted:

- **Strict tool-use schema** — the API itself guarantees the response matches the JSON schema exactly (right types, required fields, no extras). This rules out malformed JSON entirely.
- **Zod re-validation** (defense in depth) — the same shape is re-checked in application code, independent of trusting the API's own guarantee.
- **Semantic grounding check** — the one no schema can express: every `test_title/file` pair the model references must exist, verbatim, in the deterministic ground-truth list computed directly from the report. A hallucinated test name, a merged entry, or a silently dropped failure all fail this check and the entire AI analysis is discarded — the note falls back to deterministic-only sections rather than silently shipping bad data.

This isn't asserted — it's proven by `ai-utility/verify.mjs`, which deliberately constructs bad “model output” and confirms the checks catch it. Actual output of node `ai-utility/verify.mjs`:

```
--- Case 1: well-formed, grounded analysis ---
PASS - schema validation accepts a well-formed analysis
PASS - grounding check finds no errors on real data

--- Case 2: hallucinated test name (invented by the "model") ---
PASS - schema validation still accepts it (schema alone cannot catch this)
PASS - grounding check catches the count mismatch
PASS - grounding check names the specific hallucinated test
Grounding errors reported:
  - Count mismatch: model returned 2 unexpected failures, ground truth has 1.
  - Model referenced a test not present in the input: "handles concurrent booking
    writes without a race condition" (crud.spec.js).

--- Case 3: dropped entry (model omits a real failure) ---
PASS - grounding check catches a silently dropped failure

ALL CHECKS PASSED
```

5. Before Trusting This in Production

- The note is a **triage aid, not an auto-filed ticket** — a human still reads and acts on “Needs Human Attention Now”; nothing here files, closes, or reassigns issues automatically.
- The “Possibly Fixed” section is a prompt to *verify* a bug is actually fixed, never a claim that it is — promotion out of `test.fail()` stays a deliberate, reviewed code change.
- If validation fails (schema or grounding), the AI section is dropped entirely and the note says so explicitly, rather than showing a partially-trusted analysis with no indication anything was wrong.
- Before wiring this into CI for real, I'd run it against a larger sample of historical failed runs and spot-check the `likely_cause` suggestions against what the failures actually turned out to be, to catch any systematic blind spots before trusting it unattended.